Agent 架構系統設計

Agent, Rules, Skills, Workflow

課堂活動

請4-5個同學一組，共同設計一個「飲料訂購APP」，推派一位同學上台報告

活動時間 20分鐘，

請開啟ppt，讓分組同學進行共編

假設某飲料店開發一個線上訂購APP，你會如何幫忙設計?

一、你認為市面上的APP缺乏什麼？

二、你設計的APP有哪些功能， 例如會員登入、Menu查詢？(設計3-5個)

三、畫出你的功能流程

三、要完成這個專案，你認為需要哪些角色開發，例如：程式設計師？

提示：

- 想像你是產品設計師
- 不需要寫程式
- 重點是想法清楚

傳統系統開發模式-系統開發生命週期（System Development Life Cycle）

SDLC 六大階段

1 需求分析（Requirement）
➡ 使用者要什麼？要解決什麼問題？

2 系統設計（Design）
➡ 系統怎麼做？畫流程、規劃功能

3 程式開發（Development）
➡ 工程師開始寫程式

4 測試（Testing）4 測試
➡ 找錯誤（Bug）、確認功能正常

5 部署（Deployment）
➡ 系統正式上線給使用者用

6 維護（Maintenance）
➡ 修問題、更新功能

⚠️ **傳統 SDLC 特點**

- 流程「**一步一步走**」（像瀑布）流程「 **一步一步走** 」（像瀑布）
- 前面沒想好，後面很難改
- 適合需求明確、不常變動的專案

SDLC vs Agent 開發模式

SDLC（傳統開發）

👉 **先規劃好，再一步一步做**

- 流程固定（需求 → 設計 → 開發 → 測試）
- 人工主導（工程師寫程式）
- 修改成本高（前面錯，後面很難改）
- 開發時間較長

Agent 開發模式（AI 時代）

👉 **邊做邊調整，讓 AI 一起做**

- 需求可以不完整（邊做邊修）
- AI 協助產生程式、測試、文件
- 可以快速試錯（快速迭代）
- 開發速度快很多

核心差異

項目	SDLC	Agent 模式
開發方式	線性流程	反覆迭代
主導角色	人	人 + AI
需求明確度	一開始就要清楚	可以邊做邊調整
修改成本	高	低
開發速度	慢	快

Agent 專案系統架構設計

從 0 到 1 打造飲料訂購系統

核心組件介紹: Agent 設計的四大支柱

Skills (技能)

Agent「會做什麼」。一組具體的功能或工具。

例:計算金額

Workflows (工作流)

Agent「怎麼做」。一系列步驟的先後順序。

例:點餐流程

Rules (規則)

Agent「必須遵守什麼」。系統的限制與邏輯。

例:熱飲不加珠

Agent (代理人)

Agent「是誰」。系統的人格與角色設定。

例:親切店員

核心組件 1：Skills (技能) —— Agent 的工具箱

定義與設計重點

- **能力邊界**：決定了 Agent 具體能執行哪些任務。
- **單一職責**：每個技能應專注於解決一個具體問題。
- **清晰描述**：技能的名稱與描述應讓AI 易於理解何時調用。

飲料店實戰範例

calculate_price

計算飲料總金額（含加料、折扣）。

check_inventory

檢查珍珠、鮮奶等原料是否充足。

send_order

將確認後的訂單傳送到廚房系統。

Q1 想一想

你規畫的功能中，除了下列的skill外，還需要什麼skill?

飲料店實戰範例

calculate_price

計算飲料總金額（含加料、折扣）。

check_inventory

檢查珍珠、鮮奶等原料是否充足。

send_order

將確認後的訂單傳送到廚房系統。

核心組件 2：Workflows (工作流) —— Agent 的行動指南

飲料店點餐標準流程 (SOP)

STEP 01	STEP 02	STEP 03	STEP 04	STEP 05
詢問品項	確認細節	計算金額	確認付款	傳送訂單

邏輯連貫性：確保步驟之間有明確的因果關係。

異常處理：考慮顧客改單或庫存不足時的應對流程。

Q2 想一想

你規畫的功能中，除了下面的workflow外，還需要什麼?

飲料店點餐標準流程 (SOP)

STEP 01		STEP 02		STEP 03		STEP 04		STEP 05
詢問品項	>	確認細節	>	計算金額	>	確認付款	>	傳送訂單

邏輯連貫性：確保步驟之間有明確的因果關係。

異常處理：考慮顧客改單或庫存不足時的應對流程。

核心組件 3：Rules (規則) —— Agent 的行為準則

定義與設計重點

- **行為邊界**：設定 Agent 不可逾越的底線，確保系統安全性。
- **邏輯一致性**：規則必須明確且無歧義，避免AI 產生幻覺。
- **強制執行**：規則優先於工作流，是系統運作的最高準則。

飲料店實戰規則表

規則類別	具體內容
加料限制	熱飲類品項一律不可加珍珠（避免口感變質）。
外送門檻	外送距離超過 2 公里，訂單總額需滿 300 元。
營業時間	每日 21:00 後停止接收新訂單，僅處理已成立訂單。
甜度限制	鮮奶茶系列最低僅提供「微糖」，不提供「無糖」。

Q3 想一想

你規畫的功能中，除了上述Rules外，還需要什麼?

飲料店實戰規則表

規則類別	具體內容
加料限制	熱飲類品項一律不可加珍珠（避免口感變質）。
外送門檻	外送距離超過 2 公里，訂單總額需滿 300 元。
營業時間	每日 21:00 後停止接收新訂單，僅處理已成立訂單。
甜度限制	鮮奶茶系列最低僅提供「微糖」，不提供「無糖」。

核心組件 4：Agent (代理人) —— 系統的人格設定

定義與設計重點

- **角色背景**：定義 Agent 的身份、專業領域與權限。
- **語氣風格**：決定 Agent 與用戶互動時的態度（如：親切、專業）。
- **核心任務**：明確 Agent 最終要達成的目標是什麼。

飲料店店員 Prompt 範例

SYSTEM PROMPT

*「你是一位**親切且專業** 的飲料店店員。你的任務是引導顧客完成點餐流程。在對話中，請務必確認顧客的**甜度與冰塊** 需求。如果顧客點選熱飲，請禮貌地提醒**不可加珍珠**。你的目標是確保訂單資訊準確並讓顧客感到賓至如歸。」*

討論時間 20分鐘

請把Q1 - Q3的答案填至PPT或Word中

各組派員上台報告

Agent MD 檔寫作建議

2026 年的 Agent 設計文件不僅要讓人讀懂，更要讓機器能「理解」並自動化處理。

一、機器可讀性與結構化數據

- **YAML Front Matter**
  在 .md 檔案開頭使用 YAML 格式定義元數據 (Metadata)，例如 id、name、version、author、description 等。這有助於自動化工具解析文件資訊。
- **標準化表格**
  對於 Skills 和 Rules 等需要定義多個屬性的內容，建議使用 Markdown 表格，並確保欄位名稱一致，以便程式自動提取資訊。
- **明確的輸入/輸出格式**
  在描述 Skills 或 Workflows 時，應明確指出其預期的輸入 (Input) 和輸出 (Output) 數據格式，例如 JSON Schema 或具體數據類型。

二、版本控制與可追溯性

- 版本資訊每個 .md 檔案應包含 version 字段，記錄文件的版本號，便於追蹤變更。
- 作者與日期：記錄 author 和 date，明確文件的負責人與最後更新時間。
- 變更日誌：建議在文件末尾或獨立的 CHANGELOG.md 中記錄重要的變更歷史。

三、原子性與模組化

- **單一職責原則：**
  每個 .md 檔案應專注於定義一個單一的 Agent 組件（例如，一個 Skill、一個 Workflow），避免將多個不相關的內容混雜在同一個文件中。
- **引用與連結**
  當一個組件需要引用另一個組件時，應使用明確的 ID 或連結，而非重複描述。

四、清晰的語義定義與範例驅動

- 精確的語言：避免使用模糊或歧義的詞語。對於關鍵術語，應提供明確的定義。
- 具體範例：每個 Skills、Rules 或 Workflow 的步驟都應提供至少一個具體的範例，展示其在實際情境中的應用。
- 錯誤處理：在 Workflows 中，應考慮並描述潛在的錯誤情況及其處理方式。

Agent 定義模板 (Agent.md)- 參考用

Markdown

```markdown
---
id: barista_agent
name: 飲料店全能店員
version: 1.0.0
author: Manus AI
description: 負責處理飲料訂單的 Agent，具備點餐、計算金額、庫存檢查等能力。
created_at: 2026-03-25
updated_at: 2026-03-25
---

# Agent: 飲料店全能店員

## 角色設定 (Role Definition)

*   **角色名稱 (Role Name)**: 飲料店全能店員 (All-in-one Barista)
*   **任務目標 (Goal)**: 協助顧客完成點餐、計算金額並確保訂單正確，提供流暢的點餐體驗。
*   **語氣風格 (Tone & Style)**: 親切、有禮貌、專業，必要時能提供建議。
*   **核心職責 (Core Responsibilities)**:
    *   引導顧客點餐。
    *   處理訂單細節（甜度、冰塊、加料）。
    *   計算訂單總金額。
    *   檢查庫存。
    *   傳送訂單至廚房。
```

```markdown
## 互動範例 (Interaction Examples)

### 範例 1: 成功點餐

*   **顧客輸入**: 「我要一杯珍珠奶茶，半糖少冰。」
*   **Agent 回應**: 「好的，一杯珍珠奶茶半糖少冰。請問需要加購其他品項嗎？」

### 範例 2: 庫存不足

*   **顧客輸入**: 「我要一杯珍珠奶茶，半糖少冰。」 (假設珍珠已售罄)
*   **Agent 回應**: 「非常抱歉，珍珠目前已經售罄了。請問您想換成其他配料，例如椰果或布丁嗎？」

## 備註 (Notes)

此 Agent 應具備獨立處理完整訂單流程的能力，無需其他 Agent 協作。
```

Skills 定義模板 (Skills.md)- 參考用

```yaml
---
id: calculate_price
name: 計算訂單金額
version: 1.0.0
author: Manus AI
description: 根據飲料品項、加料、折扣等資訊計算訂單總金額。
input_schema: # 預期輸入的 JSON Schema 或數據結構描述
  type: object
  properties:
    items:
      type: array
      items:
        type: object
        properties:
          name: {type: string, description: '飲料名稱'}
          price: {type: number, description: '單價'}
          add_ons: {type: array, items: {type: string, description: '加料名稱'}}
          add_on_prices: {type: array, items: {type: number, description: '加料價格'}}
    discounts: {type: array, items: {type: object, properties: {type: {type: string}, valu
output_schema: # 預期輸出的 JSON Schema 或數據結構描述
  type: object
  properties:
    total_amount: {type: number, description: '訂單總金額'}
    details: {type: string, description: '金額計算明細'}
---
```

```markdown
# Skill: 計算訂單金額 (`calculate_price`)

## 功能描述 (Function Description)

此技能負責根據顧客所點選的飲料品項、加料選項以及可能存在的折扣，精確計算出訂單的最終總金額。它會考
```

```markdown
## 參數 (Parameters)

| 參數名稱 | 類型 | 描述 | 是否必填 |
| :--- | :--- | :--- | :--- |
| `items` | Array of Object | 顧客點選的飲料品項列表，包含名稱、單價及加料資訊。 | 是 |
| `discounts` | Array of Object | 應用於訂單的折扣列表，包含折扣類型與數值。 | 否 |

## 輸出 (Output)

| 輸出名稱 | 類型 | 描述 |
| :--- | :--- | :--- |
| `total_amount` | Number | 訂單的最終總金額。 |
| `details` | String | 金額計算的詳細明細，例如「珍珠奶茶 50 元 + 珍珠 10 元 = 60 元」。 |
```

```markdown
## 執行範例 (Execution Example)

### 輸入 (Input)

```json
{
  "items": [
    {
      "name": "珍珠奶茶",
      "price": 50,
      "add_ons": ["珍珠"],
      "add_on_prices": [10]
    }
  ],
  "discounts": []
}
```
```

先編寫黃框的內容

Rules 定義模板 (Rules.md)-參考用

```
---
id: hot_drink_pearl_rule
name: 熱飲不可加珍珠規則
version: 1.0.0
author: Manus AI
description: 規定熱飲品項不能添加珍珠，以確保飲品品質。
category: Order Restriction
severity: High
---

# Rule: 熱飲不可加珍珠 (`hot_drink_pearl_rule`)

## 規則描述 (Rule Description)

此規則旨在確保飲料品質與顧客體驗。由於珍珠在熱飲中容易變軟，影響口感，因此系統規定所有熱飲品項均不可

## 觸發條件 (Trigger Conditions)

當顧客點選的飲料滿足以下兩個條件時觸發此規則：

1.  飲料的溫度屬性為「熱飲」。
2.  顧客要求添加的配料包含「珍珠」。
```

```
## 執行動作 (Action on Trigger)

1.  **拒絕加料**: 系統自動阻止將珍珠加入熱飲訂單。
2.  **提示 Agent**: Agent 應禮貌地告知顧客此限制，並建議其他可選配料。

## 範例 (Examples)

| 顧客點餐 | 規則判斷 | Agent 行為 |
| :--- | :--- | :--- |
| 「一杯熱珍珠奶茶」 | 觸發 | 「抱歉，熱飲不能加珍珠喔，請問您想換成其他配料嗎？」 |
| 「一杯冰珍珠奶茶」 | 不觸發 | 正常處理訂單 |

## 影響 (Impact)

*   **品質保證**: 維護飲品口感與品質。
*   **顧客滿意度**: 避免因口感不佳導致的客訴。

## 備註 (Notes)

此規則為硬性限制，Agent 無法繞過。若有特殊情況，需經由人工審核。
```

Workflows 定義模板 (Workflows.md)- 參考用

```
---
id: beverage_ordering_workflow
name: 飲料訂購流程
version: 1.0.0
author: Manus AI
description: 描述顧客從點餐到訂單完成的標準作業流程。
start_node: greet_customer
end_node: order_completed
---

# Workflow: 飲料訂購流程 (`beverage_ordering_workflow`)

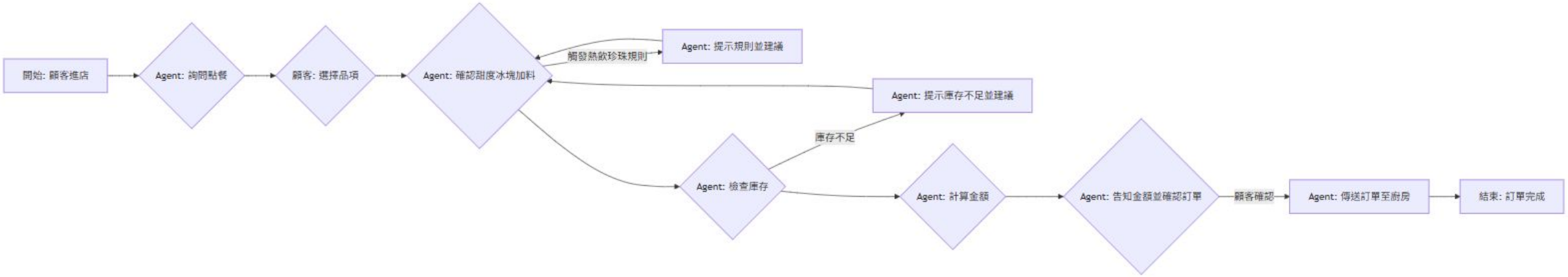
## 流程概述 (Workflow Overview)

此工作流定義了顧客在飲料店進行點餐的完整流程，從 Agent 迎接顧客開始，到最終訂單完成並傳送至廚房。它
```

```
## 流程圖 (Flow Diagram)

```mermaid
graph TD
    A[開始: 顧客進店] --> B{Agent: 詢問點餐}
    B --> C{顧客: 選擇品項}
    C --> D{Agent: 確認甜度冰塊加料}
    D -- 觸發熱飲珍珠規則 --> E[Agent: 提示規則並建議]
    E --> D
    D --> F{Agent: 檢查庫存}
    F -- 庫存不足 --> G[Agent: 提示庫存不足並建議]
    G --> D
    F --> H{Agent: 計算金額}
    H --> I{Agent: 告知金額並確認訂單}
    I -- 顧客確認 --> J[Agent: 傳送訂單至廚房]
    J --> K[結束: 訂單完成]
```
```

Workflows 定義模板 (Workflows.md)- 參考用



Workflows 定義模板 (Workflows.md)- 參考用

步驟 ID	步驟名稱	執行者	描述	相關 Skills/Rules
`greet_customer`	迎接顧客	Agent	主動向顧客打招呼，詢問點餐意願。	無
`select_item`	選擇品項	顧客	顧客選擇欲點購的飲料品項。	無
`confirm_details`	確認細節	Agent	詢問甜度、冰塊、是否加料等細節。	`hot_drink_pearl_rule`
`check_stock`	檢查庫存	Agent	調用 `check_inventory` 技能， 確認所選品項及加料的庫存。	`check_inventory`
`calculate_total`	計算金額	Agent	調用 `calculate_price` 技能，計算訂單總金額。	`calculate_price`
`confirm_order`	確認訂單	Agent	向顧客複述訂單內容與總金額，等待顧客確認。	無
`send_to_kitchen`	傳送訂單	Agent	顧客確認無誤後，調用 `send_order` 技能將訂單傳送至廚房。	`send_order`
`order_completed`	訂單完成	Agent	告知顧客訂單已送出，並感謝光臨。	無

Workflows 定義模板 (Workflows.md)- 參考用

異常處理 (Exception Handling)

- 規則觸發: 若 confirm_details 步驟觸發 hot_drink_pearl_rule, Agent 應引導顧客重新選擇配料。
- 庫存不足: 若 check_stock 步驟發現庫存不足, Agent 應引導顧客重新選擇品項或配料。
- 顧客取消: 在任何步驟, 顧客均可取消訂單, Agent 應終止當前流程。

Q4

依老師提供的範本，撰寫自己的Agent, Skill, Workflows,

Rules